



Faculty of Engineering and Technology
Department of Computer Science

COMP338-Artificial Intelligence

2025/2026

Genetic Algorithms

Instructor Name:

Abdelrahman Hamarsha

Group Members:

Monya Assi 1213503

Nahed Najjar 1220704

Contents

INTRODUCTION:	3
BACKGROUND OF GENETIC ALGORITHM:	4
Overview:	4
Steps:	5
Setting Parameters:	6
PROBLEM FORMULATION	7
Problem Definition	7
Formulation of the Problem:	7
Representation of Chromosome & Gene:	7
IMPLEMENTATION OF GENETIC ALGORITHMS:	8
PARAMETER TUNING AND ANALYSIS	10
Parameter Tuning Process	10
Plot Showing Convergence Rate	11
RESULTS AND DISCUSSION	11
Conclusion	15
REFERENCES	15

Introduction:

Genetic Algorithms (GAs) are powerful tools inspired by the process of natural selection. They are often used to solve difficult problems by finding the best solution from many possible options. In this project, we use a Genetic Algorithm to solve the problem of guessing a 32-bit password. The program creates random guesses and improves them step by step until it finds the correct password.

This project aims to understand how Genetic Algorithms work and learn how to adjust their settings to improve their performance. The report explains the steps of the algorithm, the code used, and how changes to the settings affect how quickly the algorithm finds the correct password. It also includes the results of the program and a discussion about its performance.

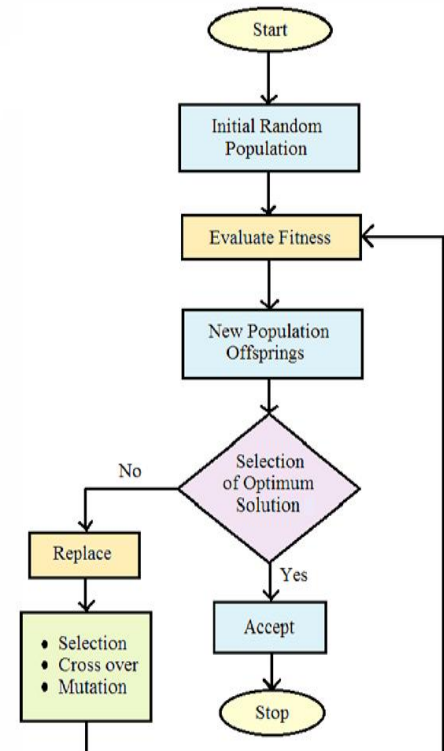
Background of Genetic Algorithm:

Overview:

- A genetic algorithm is a search technique used in computing to find true (or approximate) solutions to optimization and search problems that otherwise would take a lifetime to solve. It is frequently used to solve optimization problems, in research, and in machine learning. The GA aims to evolve a population of binary sequences (chromosomes) until one matches the 32-bit passcode. Each chromosome is a candidate solution, and each gene is a binary digit within the sequence.
- The Genetic Algorithm implemented in the code aims to find a specific passcode represented as a sequence of bits. In this case, the passcode is a binary sequence of length 32, and the algorithm simulates the process of evolving random candidate passwords (bit sequences) to match the target passcode eventually.
- In the context of this Genetic Algorithm, each potential solution (candidate password) is represented as a list of binary digits. This representation is straightforward and efficient for the algorithm to process.

Steps:

1. **Initialization:** Create a starting population of possible solutions.
2. **Evaluation:** The fitness score for a chromosome is calculated by comparing each bit of the chromosome with the corresponding bit of the target passcode. If the bits match, the fitness score is incremented using a **fitness function**. The fitness function is a crucial part of the Genetic Algorithm as it determines how "good" a candidate solution is concerning the target passcode. The fitness of each chromosome is evaluated by counting how many bits (genes) match the corresponding bits in the target passcode.
3. **Selection:** A parent is selected from the current population based on their fitness scores. Higher-fitness individuals have a higher probability of being selected for reproduction.
4. **Crossover:** This operation combines two parent chromosomes to produce offspring. A random crossover point is selected, and genes from both parents are combined to create new chromosomes (children).
5. **Mutation:** Each gene in a chromosome may undergo a mutation with a certain probability (mutation rate). This involves flipping the bit from 0 to 1 or from 1 to 0, introducing diversity into the population.
6. **Replacement:** Replace the old population with the new one.
7. **Termination:** Stop when the algorithm reaches the best solution or meets a condition.



Setting Parameters:

- 1- **Population size:** The number of solutions in each generation.
- 2- **Mutation probability:** The chance of making random changes.
- 3- **Max Generation:** The rule of ending the algorithm.

Problem Formulation

Problem Definition

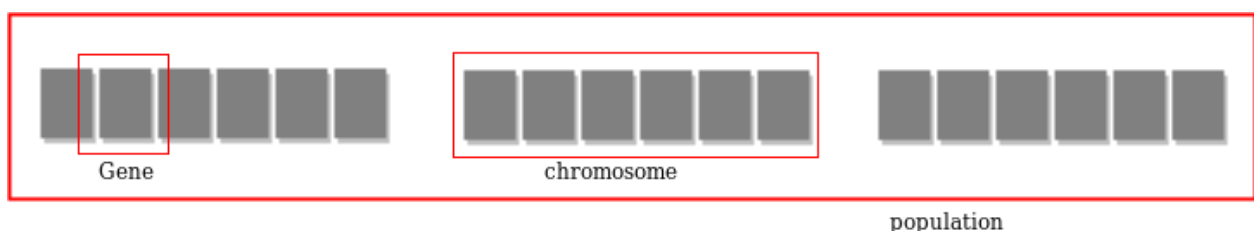
The problem is to guess a randomly generated 32-bit binary passcode by evolving a population of candidate solutions using the Genetic Algorithm. The algorithm improves the guesses over generations until the passcode is correctly found.

Formulation of the Problem:

- **Initial State:** A randomly generated population of binary sequences, each 32 bits long.
- **State:** Each state is a set of binary sequences (chromosomes) in the current generation.
- **Action:** Actions include selecting parents, applying crossover, and introducing mutations to create new sequences.
- **Transition Model:** The new population is generated by applying the Genetic Algorithm operators (selection, crossover, and mutation) to the current population.
- **Goal Test:** The goal is reached when a chromosome in the population matches the 32-bit passcode exactly.
- **Path Cost:** The cost is measured by the number of generations required and the computational time taken to find the passcode.

Representation of Chromosome & Gene:

- **Chromosome:** A chromosome represents a complete candidate solution in the code, specifically a potential guess of the target passcode. Each chromosome is a list of integers where each integer is either 0 or 1.
- **Gene:** Each gene is an individual bit within the chromosome. It represents a single component of the overall candidate solution. A gene can be either a 0 or a 1 in binary representation.



Implementation of Genetic Algorithms:

1. **Initialization:** First, makes a list of random numbers where each number is either 0 or 1.

```
public static List<Integer> generateRandomPasscode(int length) {  
    List<Integer> passcode = new ArrayList<>();  
    for (int i = 0; i < length; i++) {  
        passcode.add(random.nextInt(2));  
    }  
    return passcode;  
}
```

Then, create an initial population of solutions (chromosomes). It makes multiple lists of 0s and 1s by calling the **generateRandomPasscode** method. Each list is a potential solution.

```
public static List<List<Integer>> initializePopulation(int popSize, int chromosomeLength) {  
    List<List<Integer>> population = new ArrayList<>();  
    for (int i = 0; i < popSize; i++) {  
        population.add(generateRandomPasscode(chromosomeLength));  
    }  
    return population;  
}
```

2. **Evaluation:** Calculate the fitness of each individual in the population.

```
public static int fitnessFunction(List<Integer> candidate, List<Integer> target) {  
    int fitness = 0;  
    for (int i = 0; i < candidate.size(); i++) {  
        if (candidate.get(i).equals(target.get(i))) {  
            fitness++;  
        }  
    }  
    return fitness;  
}
```

Check how close each list in the population is to the correct passcode.

3. **Selection:** Choose individuals based on their fitness to form the next generation.

```
public static List<Integer> selectParent(List<List<Integer>> population, List<Integer> fitnessScores) {
    int totalFitness = fitnessScores.stream().mapToInt(Integer::intValue).sum();
    int randomValue = random.nextInt(totalFitness);
    int cumulativeFitness = 0;

    for (int i = 0; i < population.size(); i++) {
        cumulativeFitness += fitnessScores.get(i);
        if (cumulativeFitness > randomValue) {
            return population.get(i);
        }
    }

    return population.get(0);
}
```

Chooses one list (individual) to be a parent based on its fitness.

4. **Crossover:** Combine parts of two parents to produce offspring.

```
public static List<List<Integer>> crossover(List<Integer> parent1, List<Integer> parent2) {
    int crossoverPoint = random.nextInt(parent1.size() - 1) + 1;
    List<Integer> child1 = new ArrayList<>(parent1.subList(0, crossoverPoint));
    child1.addAll(parent2.subList(crossoverPoint, parent2.size()));
    List<Integer> child2 = new ArrayList<>(parent2.subList(0, crossoverPoint));
    child2.addAll(parent1.subList(crossoverPoint, parent1.size()));

    List<List<Integer>> children = new ArrayList<>();
    children.add(child1);
    children.add(child2);
    return children;
}
```

Combines two parent lists to create two new "children."

5. **Mutation:** Introduce random changes to maintain diversity.

```
public static List<Integer> mutate(List<Integer> chromosome, double mutationRate) {
    List<Integer> mutated = new ArrayList<>();
    for (int gene : chromosome) {
        if (random.nextDouble() < mutationRate) {
            mutated.add(1 - gene);
        } else {
            mutated.add(gene);
        }
    }
    return mutated;
}
```

Randomly changes gene in a list to introduce variety.

6. **Termination:** Stop the process once a stopping criterion is met (e.g., a maximum number of generations or desired fitness level).

```
    }  
    int maxFitness = fitnessScores.stream().mapToInt(Integer::intValue).max().orElse(0);  
    if (maxFitness == chromosomeLength) {  
        return generation;  
    }  
  
    // System.out.println(bestFitness);  
  
    List<List<Integer>> nextGeneration = new ArrayList<>();  
    while (nextGeneration.size() < popSize) {  
        List<Integer> parent1 = selectParent(population, fitnessScores);  
        List<Integer> parent2 = selectParent(population, fitnessScores);  
        List<List<Integer>> children = crossover(parent1, parent2);  
        nextGeneration.add(mutate(children.get(0), mutationRate));  
        nextGeneration.add(mutate(children.get(1), mutationRate));  
    }  
    population = nextGeneration;  
}
```

Creates a new population of individuals by combining and mutating the best ones from the current population and stops when finding the desired passcode or maximum number of generations.

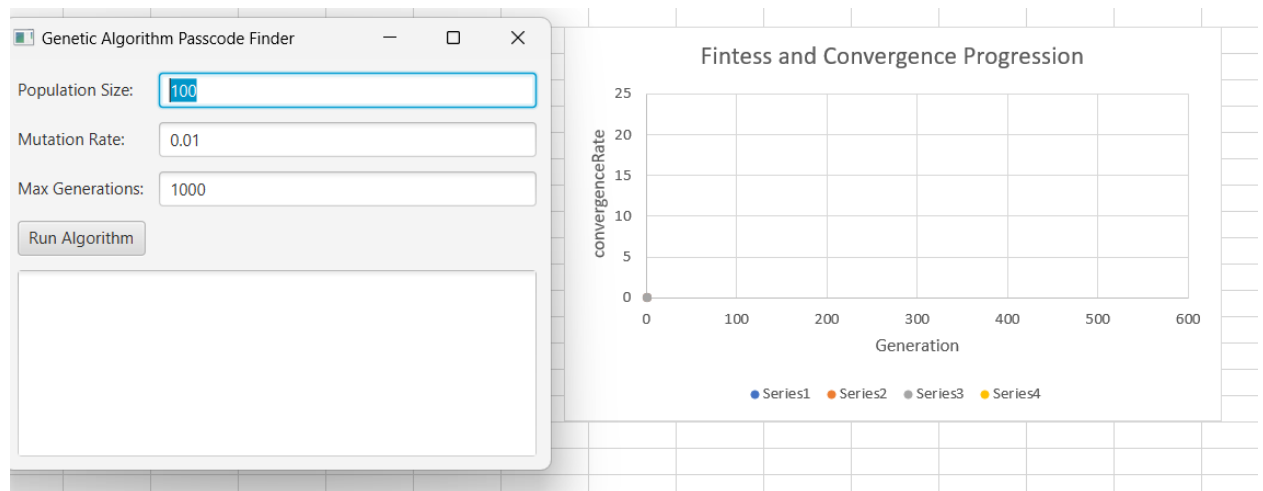
Parameter Tuning and Analysis

Parameter Tuning Process

The algorithm was tested with varying population sizes, mutation rates, and maximum generations to determine their effects on convergence.

Plot Showing Convergence Rate

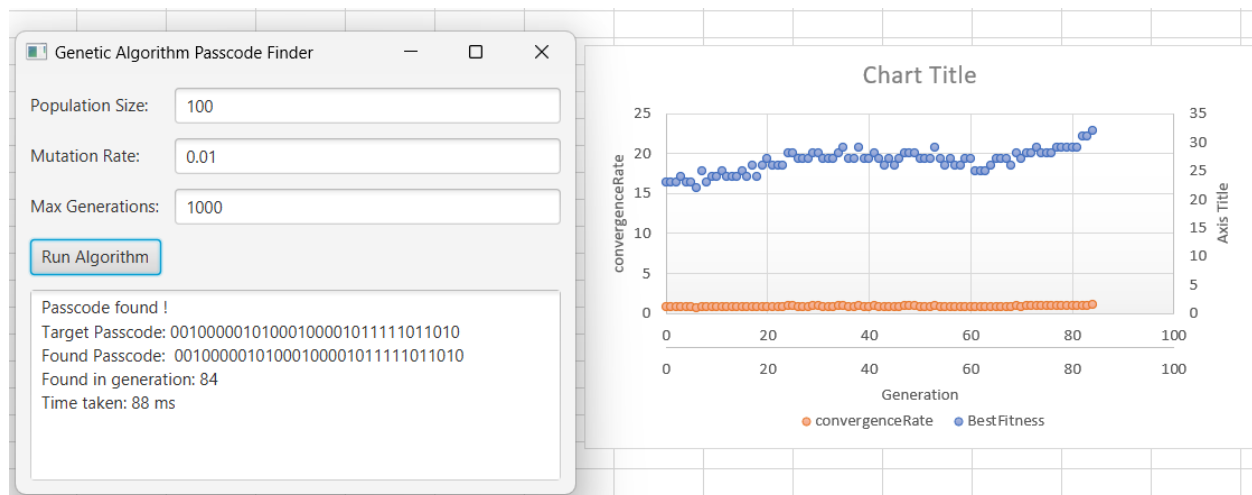
A convergence plot compares the effect of parameter settings on the number of generations required to find the passcode.



Results and Discussion

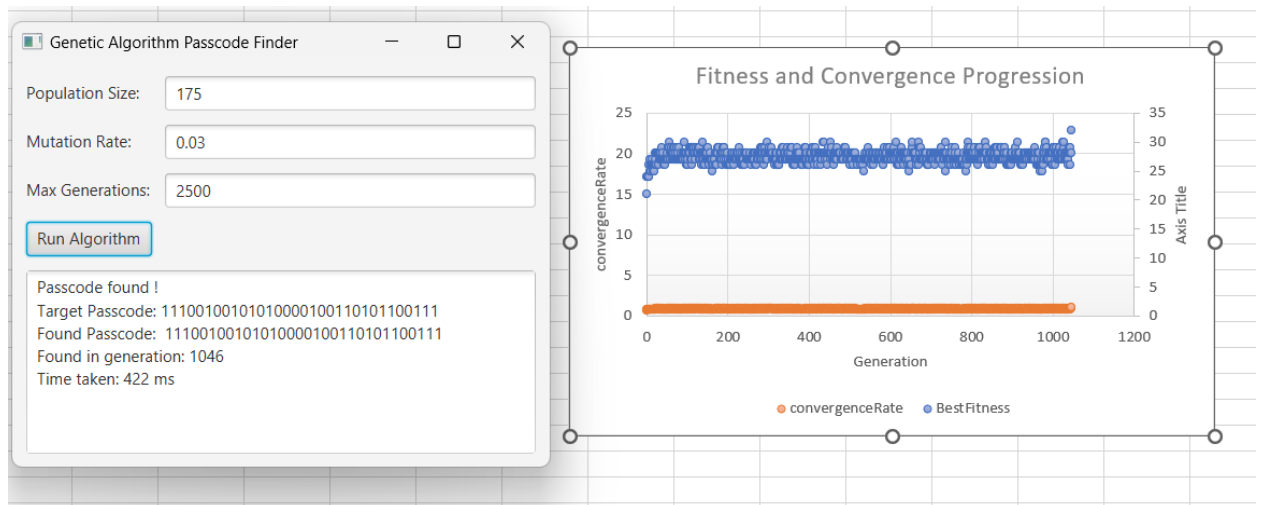
After running the algorithm, it will first create a file that show and track the generations to reach the global convergence rate.

Using JavaFX to write the interface for breaking the passcode and finding the required generations and time to find the passcode And using the Excel to draw the plot.



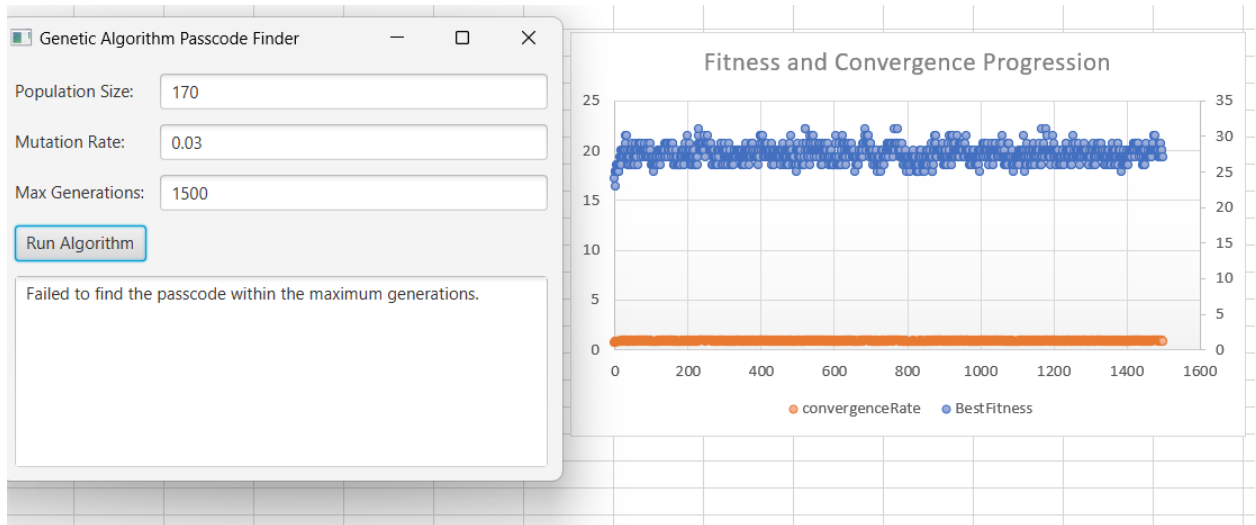
First Attempt (Successful Attempt):

In this attempt, a population size of 100 individuals with a low mutation rate of 1% was used. This reasonable population size reduced the number of computations per generation, resulting in a low execution time of approximately 88 milliseconds. The low mutation rate also helped maintain good solutions without introducing significant random changes. Consequently, the genetic algorithm was able to find the correct password in just 84 generations, demonstrating rapid and efficient convergence



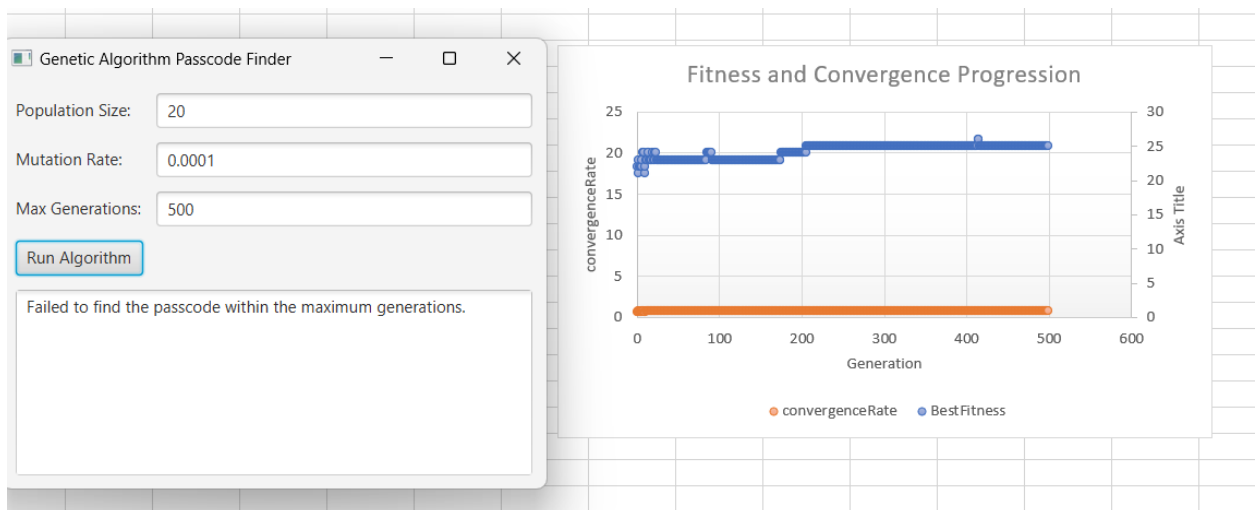
Second Attempt (Successful but Slow):

In this attempt, the population size was increased to 175 individuals, and the mutation rate was raised to 3%. The larger population size increased genetic diversity but also increased the computational cost per generation. The higher mutation rate also introduced a greater degree of randomness, which slowed down the stabilization process. As a result, the algorithm required 1046 generations to find the password, and the execution time was approximately 422 milliseconds, significantly longer than the first attempt.



Failed Attempt:

In this attempt, a population size of 170 individuals was used, with a mutation rate of 3% and a maximum of 1500 generations. Despite the large number of individuals, the relatively high mutation rate introduced frequent random changes that disabled good solutions before they could be optimized. As a result, the algorithm continued searching without clear convergence and reached the maximum number of generations (1500) without finding the password.



Failed Attempt:

In this attempt, a very small population size of 20 individuals was used, along with a very low mutation rate of 0.01% and a maximum of 500 generations. This setup resulted in a severe lack of genetic diversity, causing early convergence at local solutions. Consequently, the algorithm failed to find the correct password even after running for the full 500 generations.

Conclusion

- Larger population sizes improved the algorithm's accuracy but increased computation time.
- Higher mutation rates prevented premature convergence but disrupted convergence at excessively high values.

References

1. Local Search Slide.
2. [An Introduction to Genetic Algorithms](#)
3. [Genetic Algorithms Geeks for Geeks](#)
4. [JavaFX documentation](#)